香港中文大學(深圳)
The Chinese University of Hong Kong, Shenzhen

Week 6: Lab - Implementing and testing path planning algorithms

Shaobin Ling, Tao Huang

March 2, 2026

Table of Contents

1 Course Introduction

Path planning in Navigation.

Course Objective

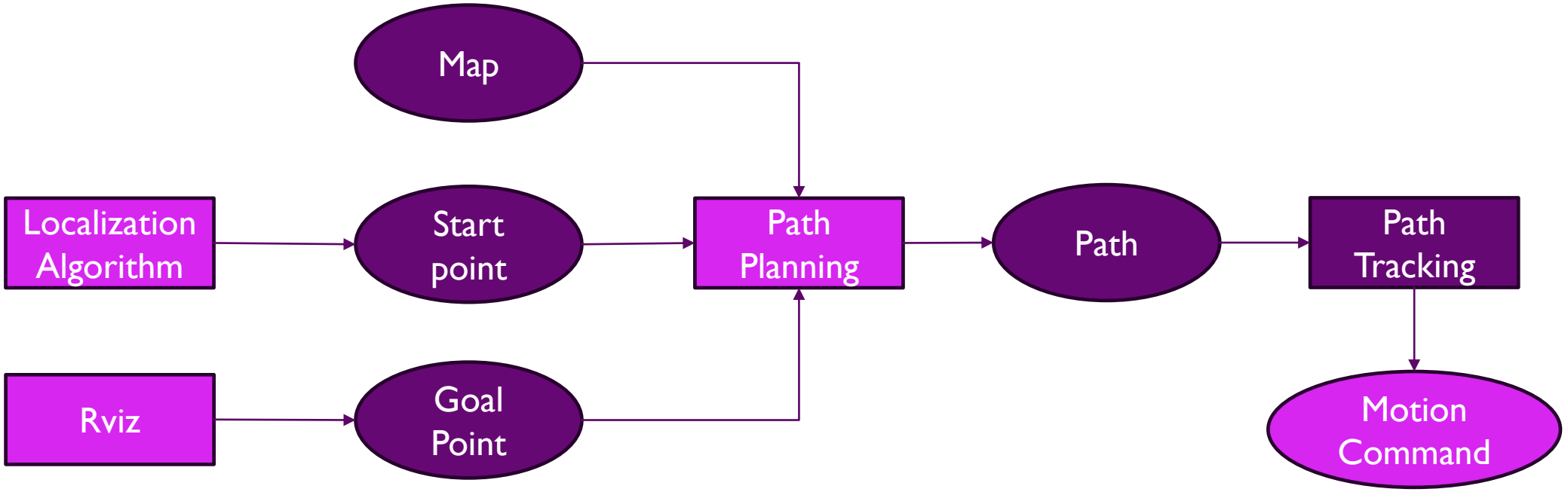
- Review path planning algorithms
- Modules included in the navigation framework
- How to modify the path planning algorithm

2 Navigation Framework Introduction

Operation procedure:

1. Determine the robot's position (starting point)
2. Determine the destination (goal point)

Navigation Framework Introduction



The Simplest Navigation Framework

Related code files:
src/lab3_path_planning/launch/path_planning_ex.launch

src/lab3_path_planning/scripts/path_planning_node.py

3 Algorithm Explanation of the Navigation Framework

How to Launch the Localization Algorithm

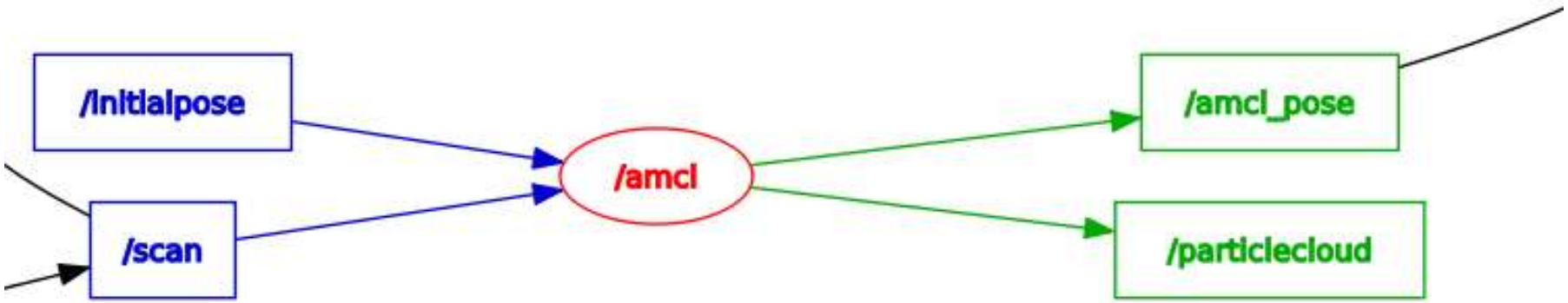
In launch file, we will launch the AMCL (Adaptive Monte Carlo Localization) localization algorithm with robot initial position.

```
<arg name="x_pos" default="-4.0"/>
<arg name="y_pos" default="-4.0"/>

<!-- AMCL -->
<include file="$(find turtlebot3_navigation)/launch/amcl.launch">
  <arg name="initial_pose_x" value="$(arg x_pos)"/>
  <arg name="initial_pose_y" value="$(arg y_pos)"/>
</include>
```

How to Launch the Localization Algorithm

In launch file, we will launch the AMCL (Adaptive Monte Carlo Localization) localization algorithm with robot initial position.

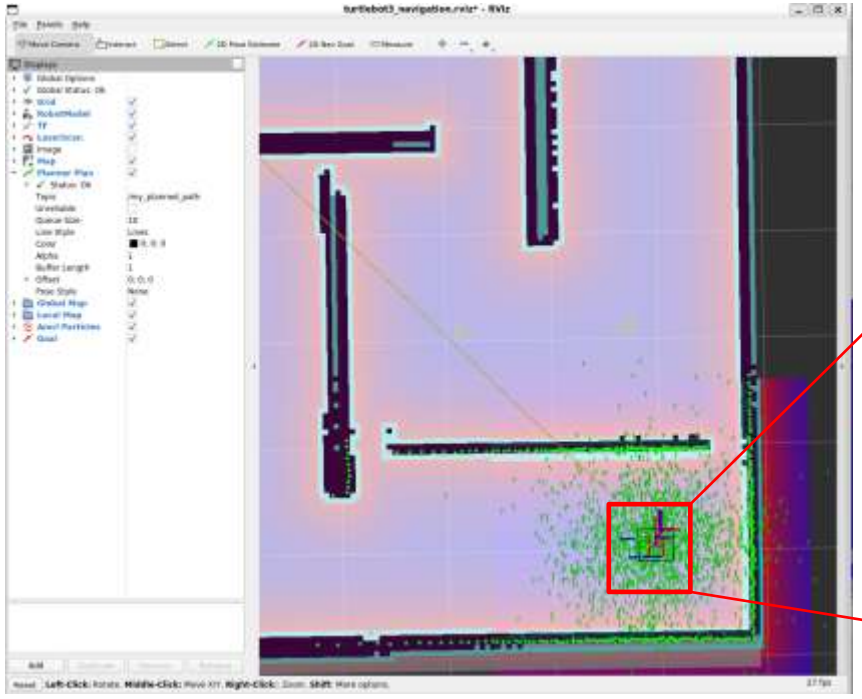


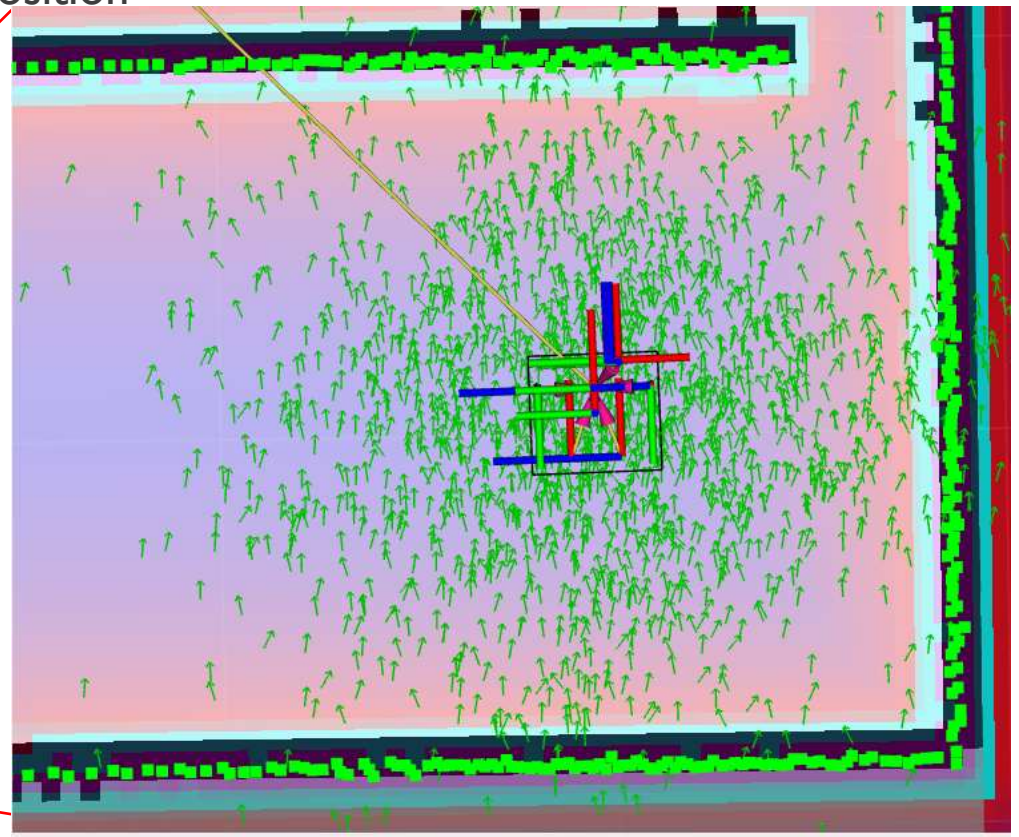
Localization Algorithm : AMCL

Adaptive Monte Carlo Localization (AMCL) is used for robot self-localization within a known map.

Initialization Phase:

- Generate a large number of particles around the robot's initial position
- Initialize each particle's weight equally





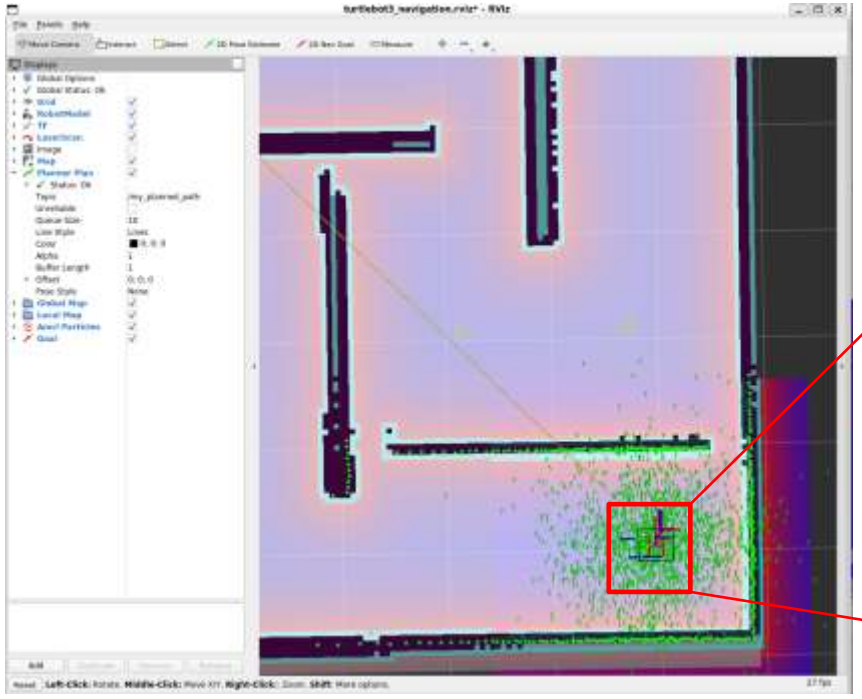
Localization Algorithm : AMCL

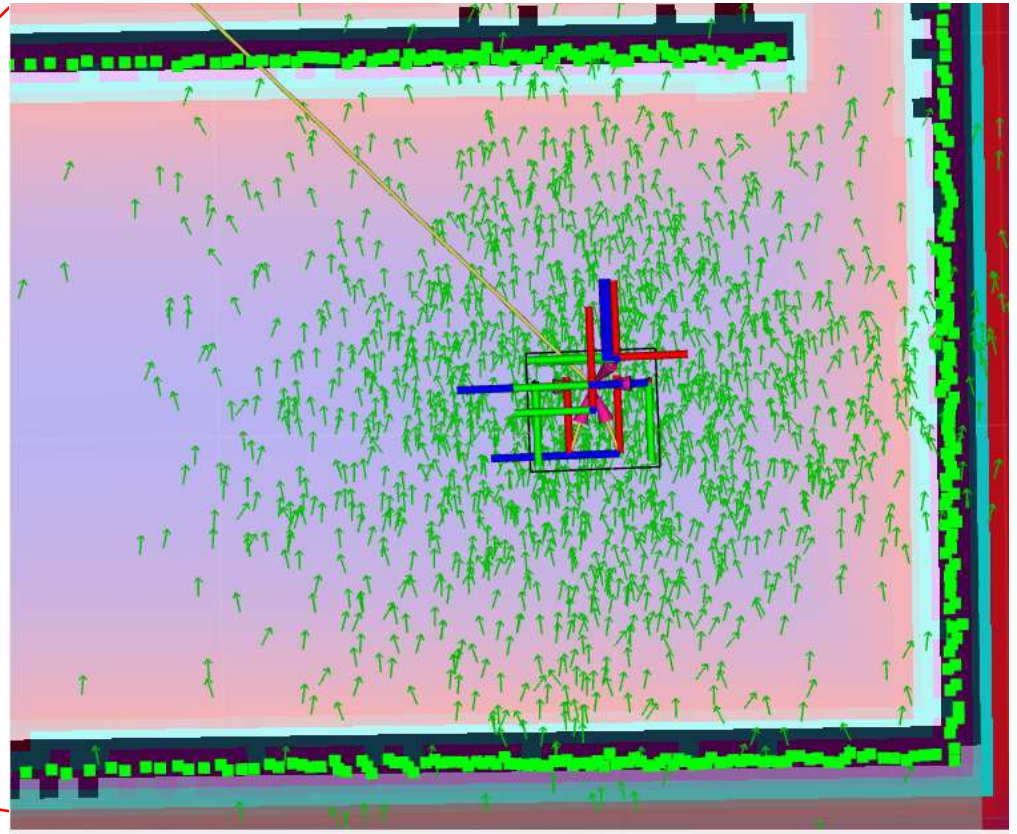
AMCL is used for robot self-localization within a known map.

Initialization Phase:

- Generate a large number of particles around the robot's initial position
- Initialize each particle's weight equally

Demostrate !





Localization Algorithm : AMCL

Prediction Phase (based on motion model): :

- Predict the new position of each particle based on previous position and velocity data

Update Phase (based on observation model):

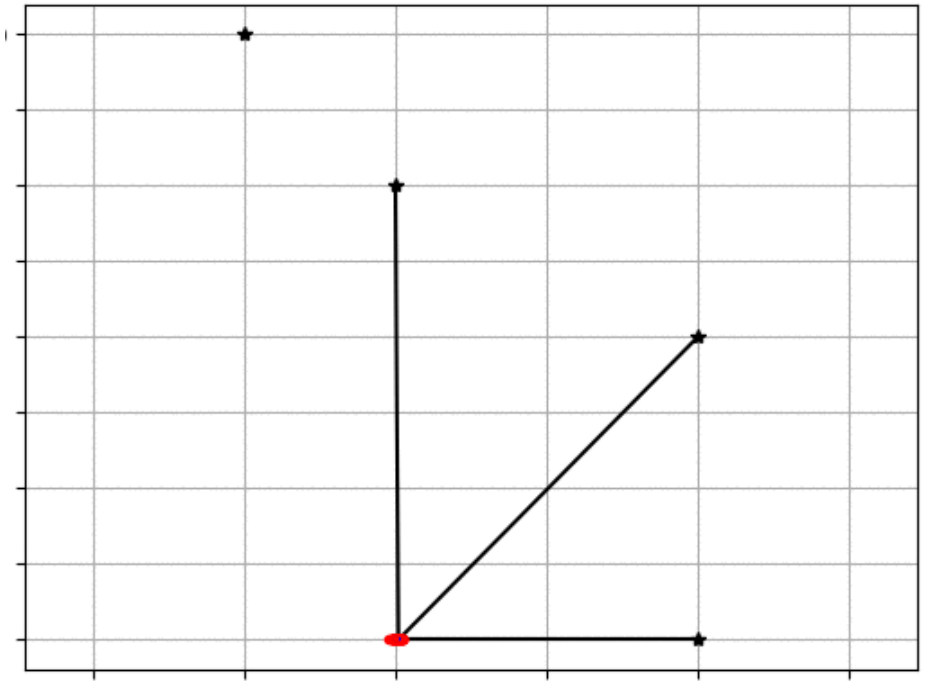
- Match LiDAR scan data against the known map

Resampling Phase:

- Resample particles according to their weights
- Particles with higher weights are duplicated;

those with lower weights are eliminated

- Adaptively adjust the number of particles

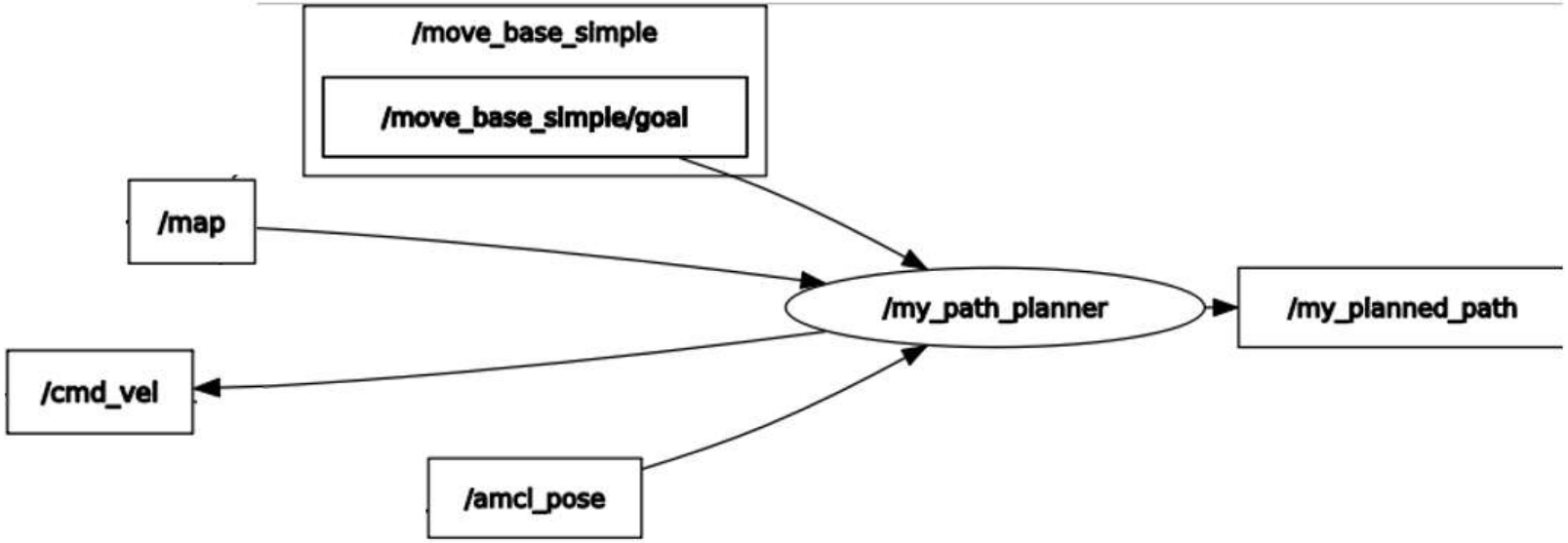


How to Launch the Path Planning Algorithm

In launch file:

```
<!-- 启动独立的规划器节点 -->
<node pkg="lab3_path_planning" type="path_planning_node.py" name="my_path_planner" output="screen"/>
```

In ros node graph:



Path Planning： straight line

Used for teaching and demonstration purposes.

The Bresenham line algorithm connects the start and goal points on a grid map, completely ignoring obstacles. Its core idea is similar to drawing a straight line on paper with a ruler: approximating an ideal line on a discrete grid.

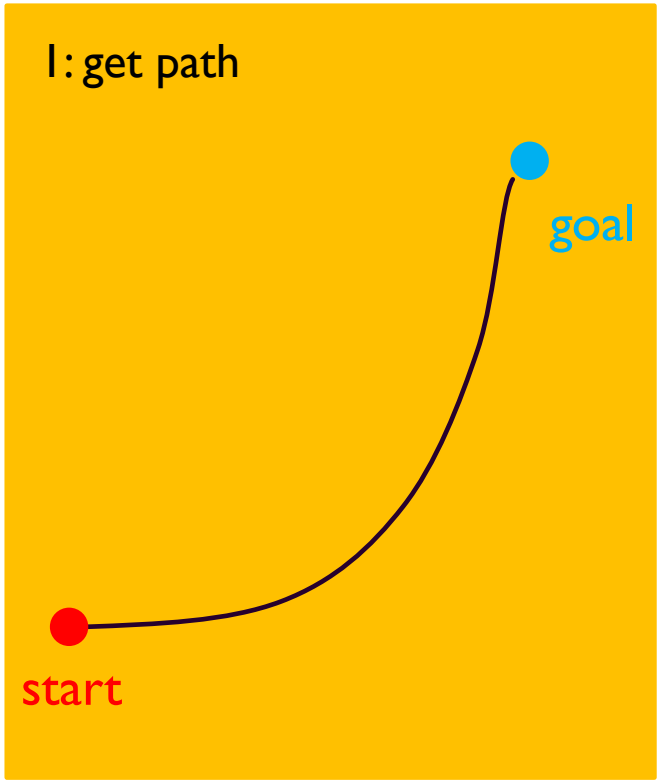
This needs to be replaced with A* or other algorithms in subsequent steps.

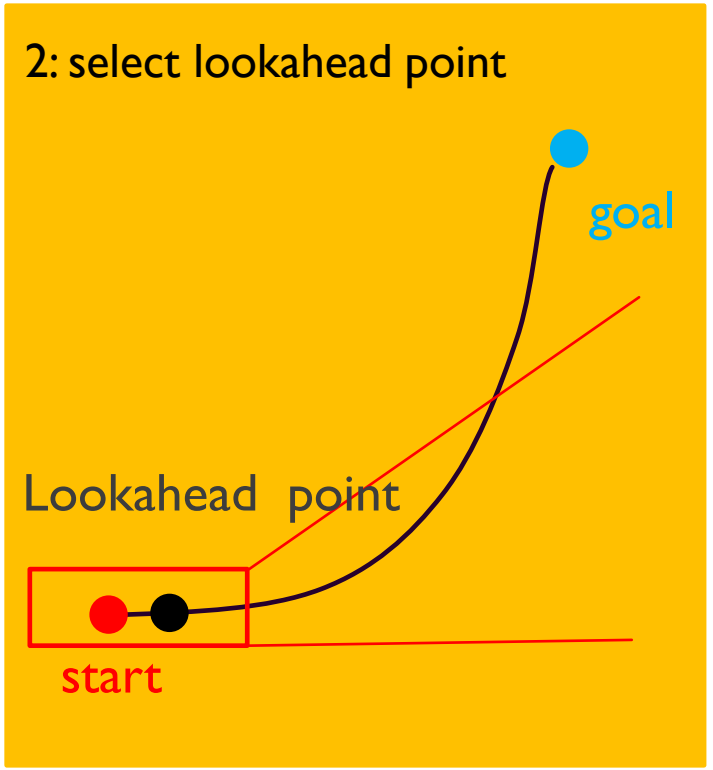
Path tracking :

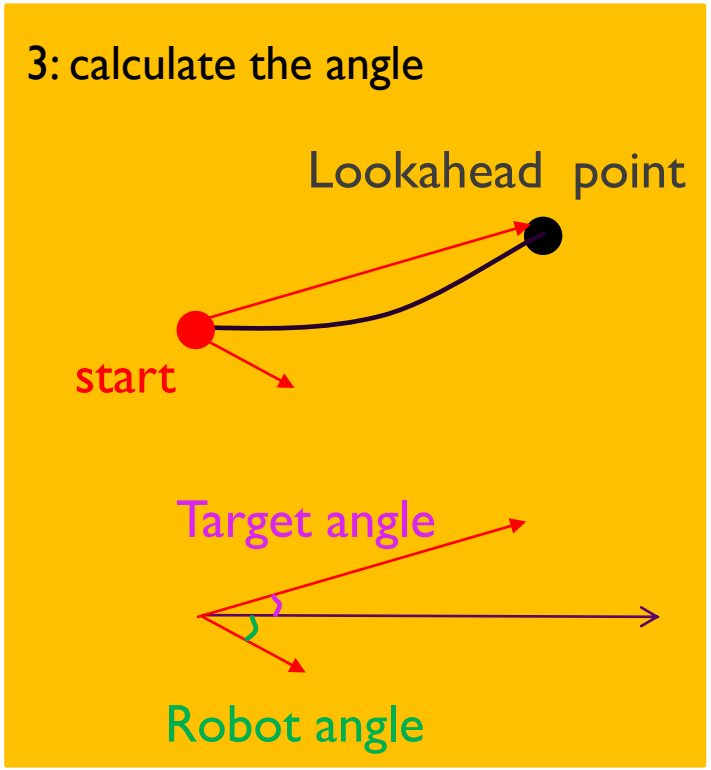
- After obtaining the planned path, we also need to convert the path into velocity commands that the robot can execute. This process is called path tracking.

Path tracking: Pure Pursuit

The Pure Pursuit algorithm is a classic geometric path tracking method.







Path tracking: Pure Pursuit

4: calculate the angle error

Robot angle

Target angle - Robot angle = Angle error

5: project angle to angle velocity

param * Angle error = Angle velocity
Constrain the absolute value of Angle velocity to be ≤ 0.5

6: calculate the linear velocity

```
if abs(angle_error) < 0.5:
        twist.linear.x = 0.2
else:
        twist.linear.x = 0.0
```

7: continue this loop until the robot arrived at the goal.

4 Modifying Path Planning

Modifying the Path Planning Algorithm

In code file: src/lab3_path_planning/scripts/path_planning_node.py : function plan_path[1]

In code file: src/lab3_path_planning/scripts/path_planning_simple.py: function plan_path[2]

Replace the function plan_path[1] with plan_path[2] or the path planning algorithm you like.

Reference website for robot algorithms:

PythonRobotics

Linux_CI passing | MacOS_CI passing | Windows_CI passing

build passing

Python codes and textbook for robotics algorithm.

Table of Contents

- What is this?
- Requirements
- Documentation
- How to use

https://github.com/AtsushiSakai/PythonRobotics/tree/master?tab=readme-ov-file#what-is-this

Reference website for robot algorithms:

- Path Planning
  - Dynamic Window Approach
  - Grid based search
    - Dijkstra algorithm
    - A* algorithm
    - D* algorithm
    - D* Lite algorithm
    - Potential Field algorithm
- Path Tracking
  - move to a pose control
  - Stanley control
  - Rear wheel feedback control
  - Linear–quadratic regulator (LQR) speed and steering control
  - Model predictive speed and steering control
  - Nonlinear Model predictive control with C-GMRES

https://github.com/AtsushiSakai/PythonRobotics/tree/master?tab=readme-ov-file#what-is-this

Q&A

Thank you!